

Kapitel 17 – Smells and Heuristics: Regeln für Clean Code

Eine praktische Fallstudie am Beispiel der "Fashion School App"

DAVID EDWARD SSEBULIBA

KURZFASSUNG

Diese Ausarbeitung demonstriert die praktische Anwendung der in Kapitel 17 von Robert C. Martins „Clean Code“ vorgestellten Regeln, Heuristiken und „Smells“. Anhand des Full-Stack-Projekts „The Fashion School App“ wird gezeigt, wie diese Prinzipien zur systematischen Verbesserung von Code-Qualität und Lesbarkeit beitragen. Der Fokus liegt auf der Umsetzung der Konzepte zu Kommentaren, Funktionsdesign und Namensgebung, um technische Schulden zu reduzieren und die Wartbarkeit zu erhöhen.

1. EINLEITUNG

Das Ziel von Clean Code ist es, Software so zu schreiben, dass sie nicht nur funktioniert, sondern auch effizient gewartet und von anderen Entwicklern verstanden werden kann. Kapitel 17 dient als Zusammenfassung vieler Beobachtungen, die während der Entwicklung entstehen und als Leitfaden für professionelles Handeln dienen.

Diese Ausarbeitung verbindet die theoretischen Heuristiken mit der praktischen Implementierung im Projekt „The Fashion School App“. In den folgenden Abschnitten werden zentrale Problemfelder wie der Umgang mit Kommentaren, das Design von Funktionen und die Bedeutung der Namensgebung analysiert und mit Code-Beispielen aus dem Projekt illustriert.

2. KOMMENTARE UND UMGEBUNG

Kommentare sollten nur dann eingesetzt werden, wenn sie einen echten Mehrwert bieten. Oft sind sie jedoch ein Zeichen für mangelnde Klarheit im Code selbst.

2.1 Umgang mit Kommentaren

Drei wesentliche Problemfelder wurden im Projekt aktiv vermieden:

- **Unangemessene Informationen:** Metadaten wie Autorennamen gehören in die Quellcodeverwaltung (Git), nicht in den Code.
- **Veraltete Kommentare:** Kommentare, die „lügen“, wurden konsequent vermieden, indem sie bei jeder Code-Änderung aktualisiert oder entfernt wurden.
- **Auskommentierter Code:** Nicht mehr benötigter Code wurde gelöscht, da die Git-Historie als Sicherung dient.

Stattdessen wurde das Prinzip der **aussagekräftigen Namen** verfolgt:

Schlechtes Beispiel (erfordert Kommentar):

```
// Prüft, ob der User die Rolle 'Lehrer' hat  
public boolean check(String username) { ... }
```

Gute Umsetzung im Projekt (UserService.java):

```
// Der Name ist selbsterklärend  
public boolean isTeacher(String username) { ... }
```

2.2 Systemumgebung (Environment)

Ein professioneller Build-Prozess erfordert einen **Build in einem Schritt**. Dies wurde im Projekt umgesetzt:

- Backend: `mvn spring-boot:run`
- Frontend: `npx react-native run-ios`

3. FUNKTIONEN UND ALLGEMEINE PRINZIPIEN

3.1 Heuristiken für Funktionen

Clean Code fordert kleine, fokussierte Funktionen mit möglichst wenigen Argumenten.

Beispiel aus dem BlogPostService.java:

```
public void deletePost(Long postId, String currentUsername) {  
    // 1. Post finden  
    BlogPost post = blogPostRepository.findById(postId)  
        .orElseThrow(() -> new EntityNotFoundException(...));  
  
    // 2. Berechtigung prüfen (Ownership)  
    if (!post.getUsername().equals(currentUsername)) {  
        throw new AccessDeniedException(...);  
    }  
  
    // 3. Post und zugehörige Datei löschen  
    blogPostRepository.deleteById(postId);  
    deleteAssociatedImage(post.getImagePath());  
}
```

Analyse: Die Funktion hat einen klaren Namen (`deletePost`), nur zwei notwendige Argumente und keine `boolean`-Flags.

3.2 Allgemeine Entwurfsprinzipien

Ein wichtiges Prinzip ist die Vermeidung von "**verdunkelter Absicht**". „Magic Numbers“ oder „Magic Strings“ sollten durch benannte Konstanten oder Enums ersetzt werden.

Beispiel aus dem Projekt: Aufgaben-Level

Schlechter Ansatz ("Magic String"):

```
// Was bedeutet "B"? Unklar und fehleranfällig.  
if (assignment.getLevel().equals("B")) { ... }
```

Unsere saubere Lösung mit einem Enum:

```
// Level.java  
public enum Level {  
    BEGINNER, ADVANCED, EXPERT  
}  
  
// Assignment.java  
@Enumerated(EnumType.STRING)  
private Level level;
```

Durch das `enum` ist der Code selbsterklärend und typsicher.

4. JAVA-SPEZIFIKA UND NAMENSgebung

Namen machen etwa 90% der Lesbarkeit aus. Ein gutes Beispiel im Projekt ist die strikte Einhaltung von **Abstraktionsebenen** in der 3-Schichten-Architektur (Controller, Service, Repository), bei der jede Schicht eine klar benannte Verantwortung hat.

5. FAZIT

Die Anwendung der Clean-Code-Heuristiken führte im Projekt „The Fashion School App“ zu einer signifikanten Reduktion technischer Schulden und verbesserte die Wartbarkeit. Die größte Herausforderung bleibt die Disziplin, diese Standards konsequent einzuhalten, was letztlich zu einem professionelleren und langlebigeren Softwareprodukt führt.

LITERATUR

Martin, Robert C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.